Camunda Signal Event

In this session we will look at what is signal event and how it functions ?.

Modeling a business process may require some kind of communication with other related business processes. The reason for this may be to wake up a process that was waiting for a particular situation to verify or to start a new process. The BPMN notation provides two different concepts for this purpose: the Message and the Signal. Both of them are events.

The main difference is that a Message is triggered to a single target while a Signal is broadcast to multiple targets. In other words, the Message is a point-to-point communication while the Signal is a broadcast communication.

- Signals are commonly used for general communication within and across Process Levels, across Pools and between business process diagrams.
- A Signal is composed of a name and a set of parameters. These parameters are sent when the signal is thrown; otherwise, they are received when the signal is caught. As the Signal is broadcast, different processes can have catching events listening for the same signal. Of course, there's also the chance of a process throwing a signal.
- Signal events are events which reference a named signal. A signal is an event of global scope (broadcast semantics) and is delivered to all active handlers.
- A Signal Event provides the facility of loosely coupling 'throwers' and 'catchers' by publish-subscribe integration. A 'thrower' will broadcast a signal rather than addressing it to a particular process; any process listening for that particular event could trigger a new instance using a Signal Start Event.
- A Signal can be thrown from either a throwing Intermediate Event or a throwing End Event, and can be caught in a Start Event or a catching Intermediate Event (including a boundary Signal Event).

Signal event types:

1. Start Signal Event.
2. Intermediate Throwing Signal Event.

3. Intermediate Catching Event.
4. Boundary Signal Event.

Start signal event

A signal start event can be used to start a process instance using a named signal.

When deploying a process definition with one or more signal start events, the following considerations apply:

- The name of the signal start event must be unique across a given process definition, i.e., a process definition must not have multiple signal start events with the same name. The engine throws an exception upon deployment of a process definition in case two or more signal start events reference the same signal or if two or more signal start events reference signals with the same signal name.
- Contrary to message start events, the name of the signal start event does not have to be unique across all deployed process definitions.
- Process versioning: Upon deployment of a new version of a process definition, the signal subscriptions of the previous version are canceled. This is also the case for signal events that are not present in the new version.

  A process instance of a process definition with one or more signal start events will be started, when a signal with the proper name is thrown. The signal can either be thrown by a process instance (i.e., on intermediate throwing signal event or signal end event) or using the methods on the RuntimeService:

Intermediate throw signal event:

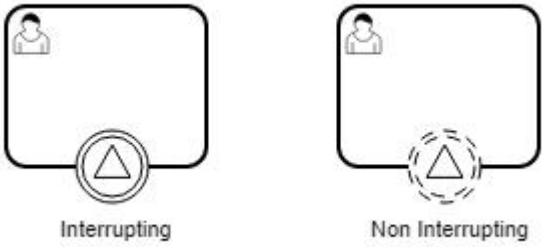
A signal intermediate throwing event throws a signal event for a defined signal.

- The signal is broadcast to all active handlers (i.e., all catching signal events). Signals can be published synchronously or asynchronously.
- In the default configuration, the signal is delivered synchronously. This means that the throwing process instance waits until the signal is delivered to all catching process instances. The catching process instances are also notified in the same transaction as the throwing process instance, which means that if one of the notified instances produces a technical error (throws an exception), all involved instances fail.
- A signal can also be delivered asynchronously. In that case it is determined which handlers are active at the time the throwing signal event is reached. For each active handler, an asynchronous notification message (Job) is stored and delivered by the JobExecutor.

Intermediate catch signal event:

The Intermediate Catching Signal Event is an event that waits for the associated Signal to be thrown.

Boundary signal event:



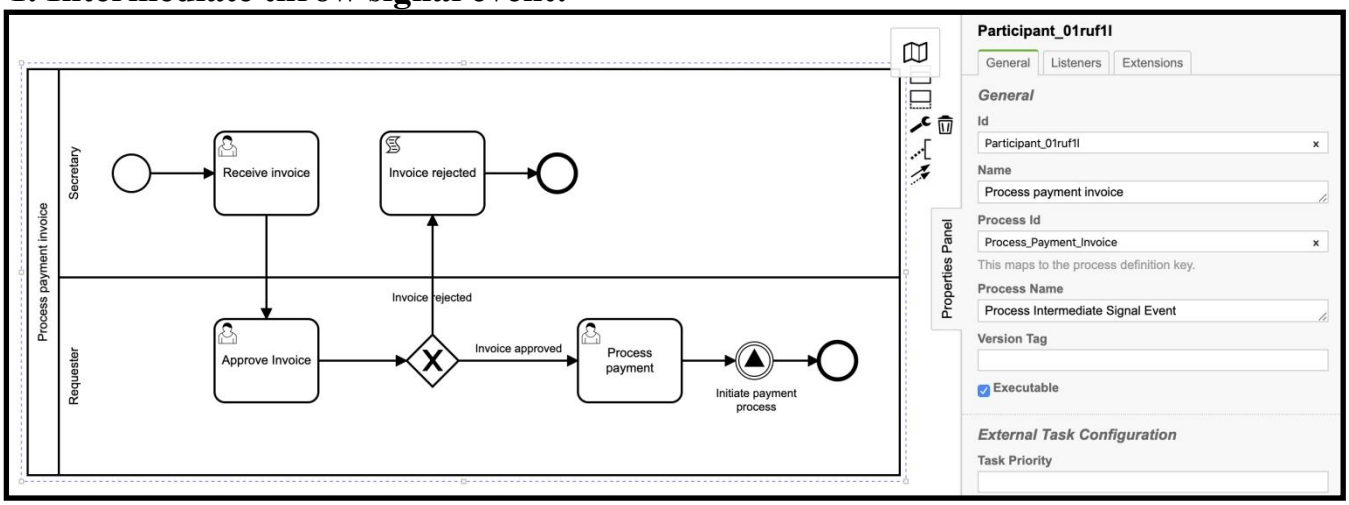
A boundary signal event listens to a signal being fired (from within the process instance or system-wide) while the activity upon which the event is defined is active.

- When an execution arrives in the activity to which the signal boundary event is attached, the signal boundary event catches signals with the proper name.
- Note: Contrary to other events, such as the error boundary event, a signal boundary event does not only catch signal events thrown from the scope it is attached to. A signal event has a global scope (broadcast semantics), meaning that the signal can be thrown from any place, even from a different process instance.

We will demonstrate the use of intermediate throw signal event and start signal event using below processes.

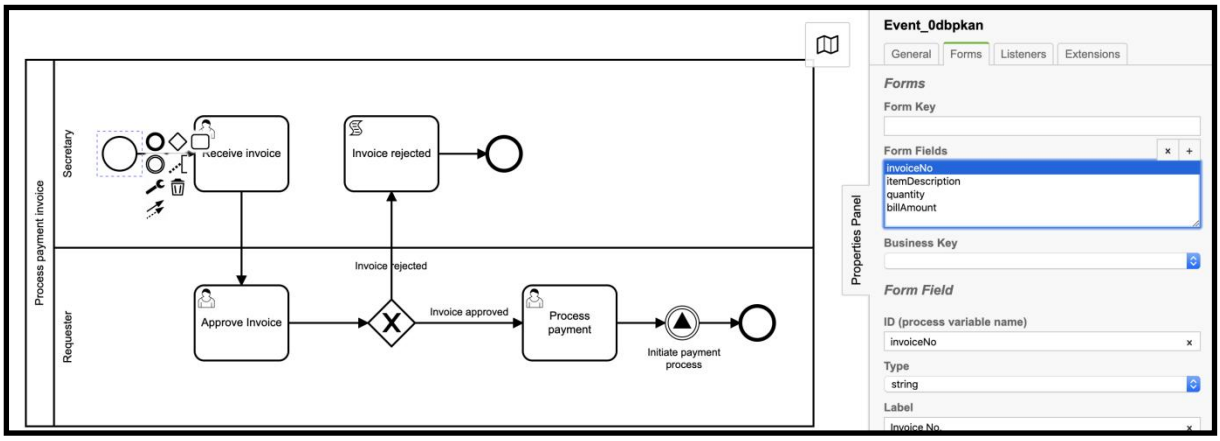
As per process invoice is received by secretary and then it will pass to requester. Requester will approve or reject the invoice provided by secretary. If invoice is rejected then rejection notification will be generated else process payment user task will be get created and then signal will be thrown to process payment. Same thrown signal is caught by signal start event and check will be generated and then bank transfer will happen.

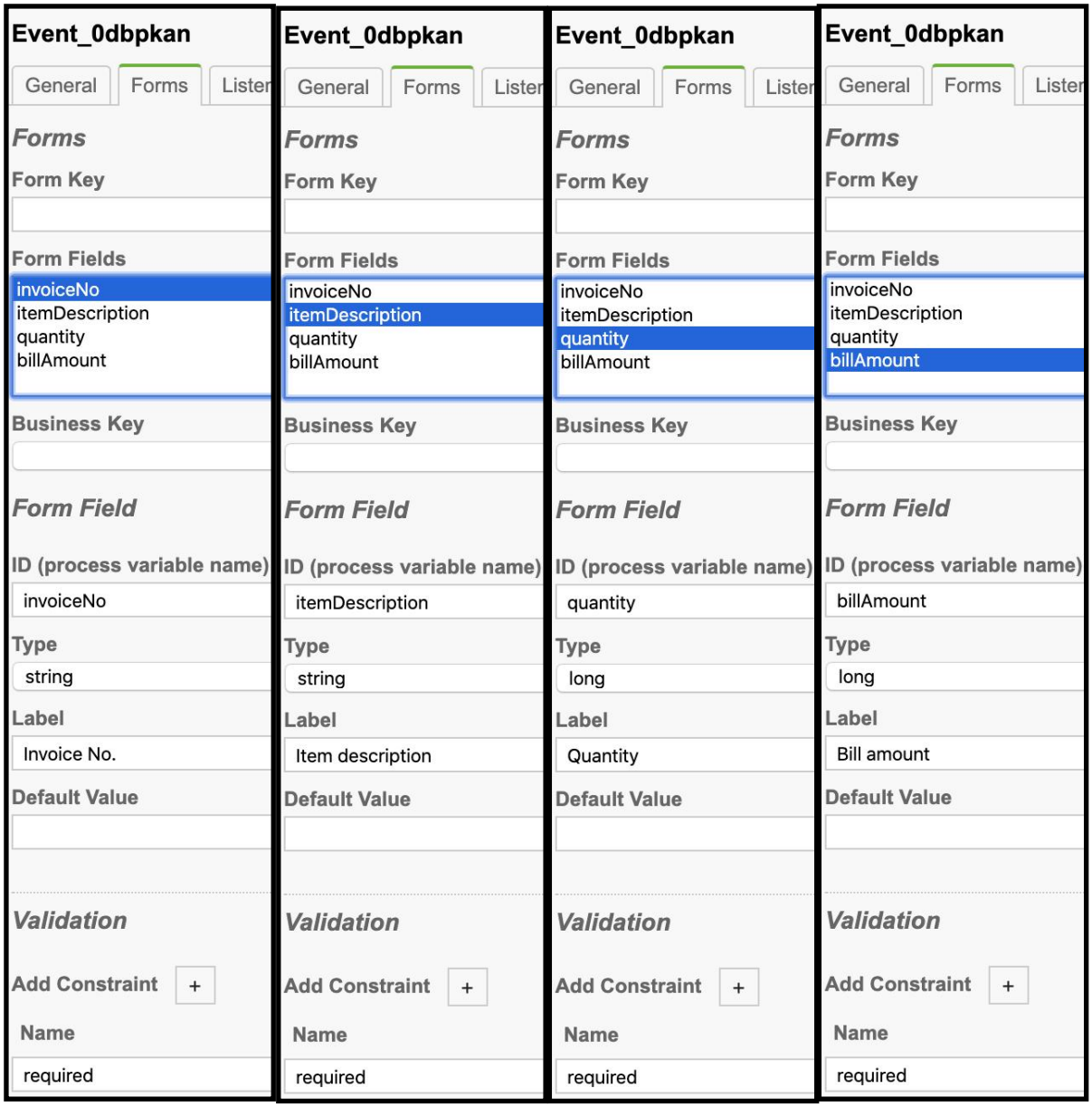
1. Intermediate throw signal event:



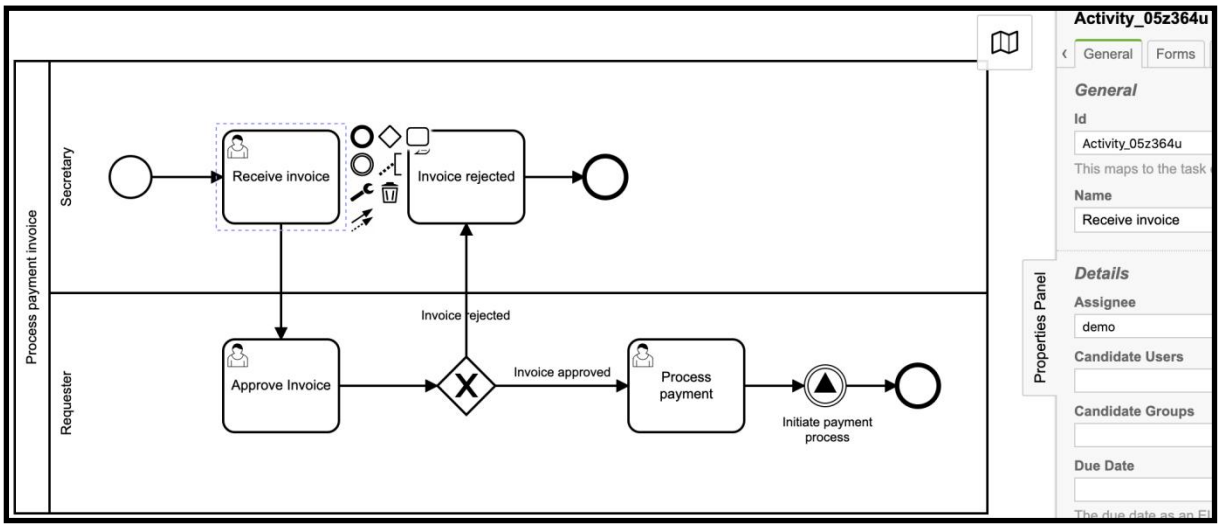
Id: Participant_01ruf1l
Name: Process payment invoice
Process Id: Process_Payment_Invoice
Process Name: Process Intermediate Signal Event

Start event form:



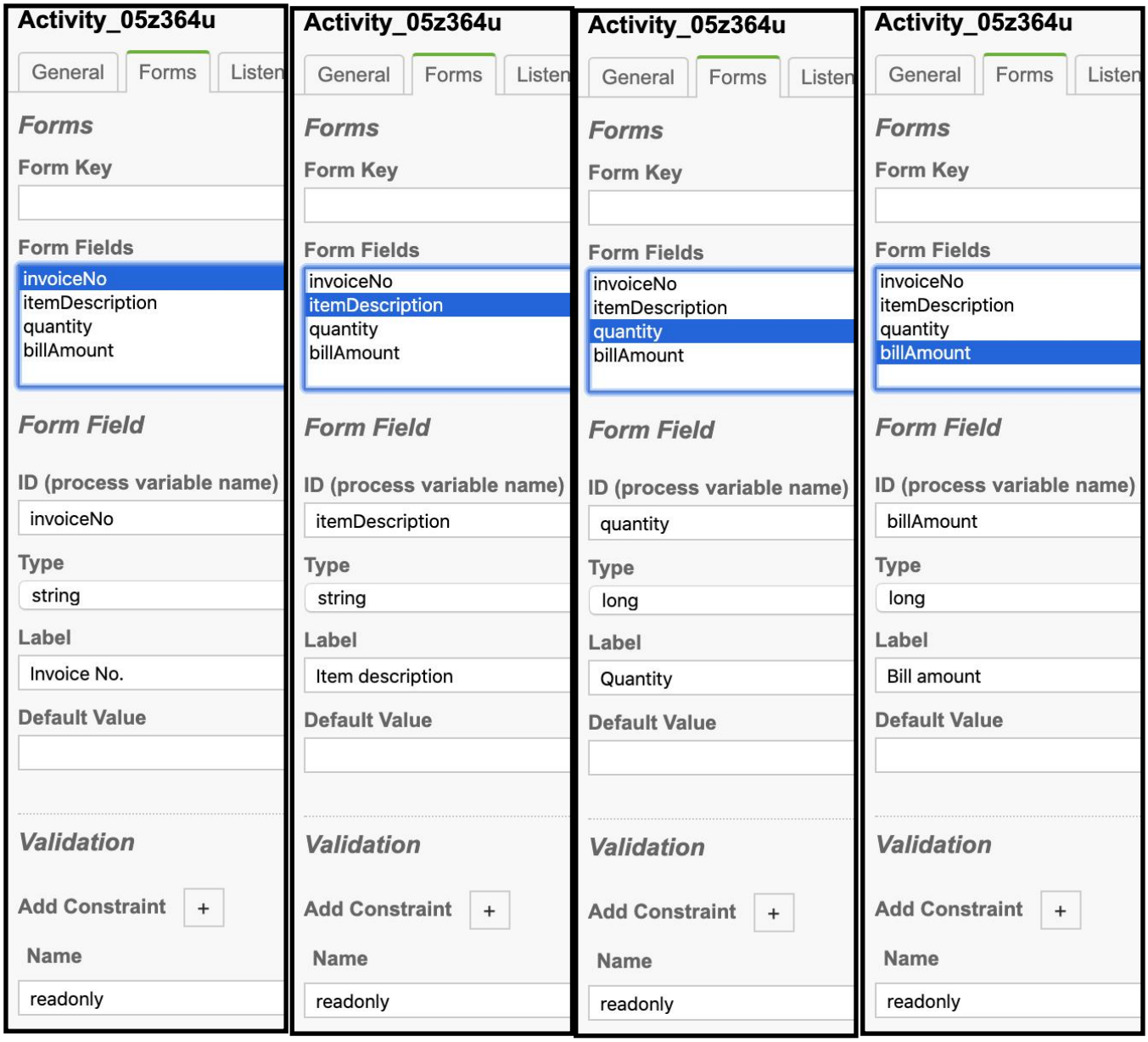


User task: Receive Invoice

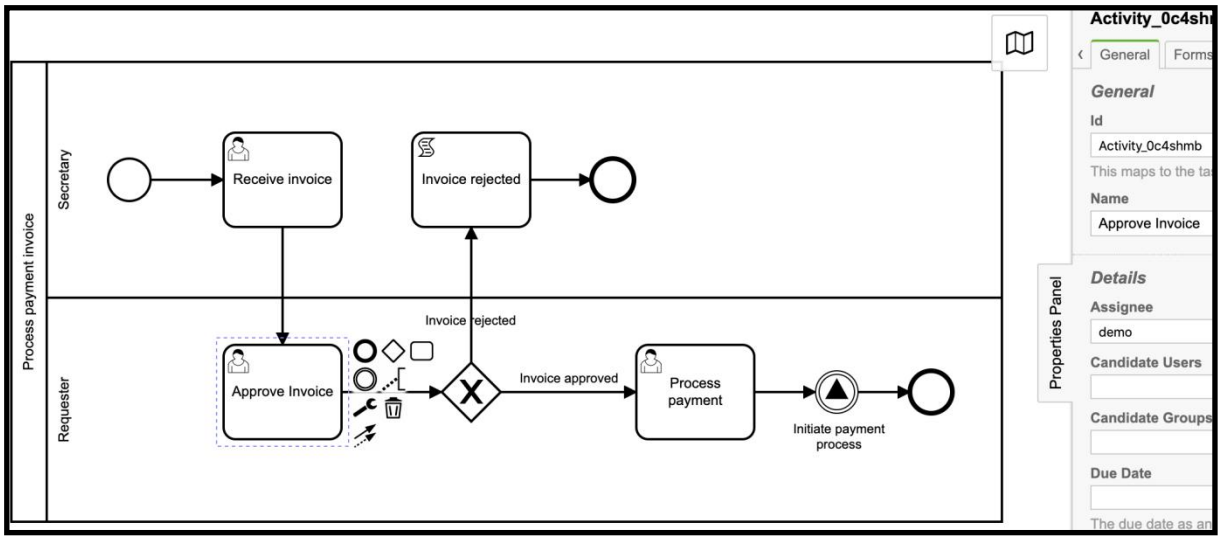


Id: Activity_05z364u
Name: Receive invoice
Assignee: demo

Task form:

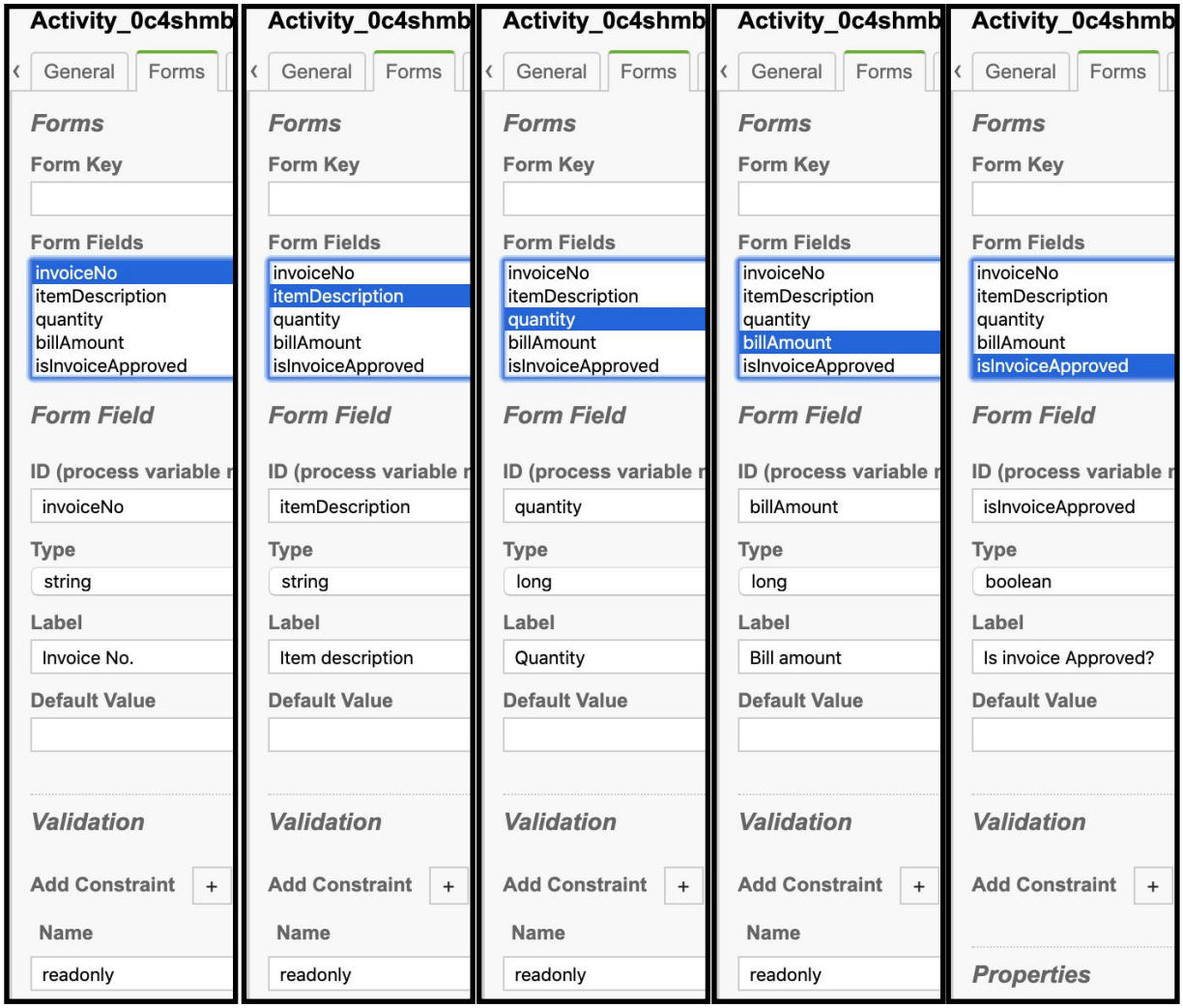


User task: Approve Invoice

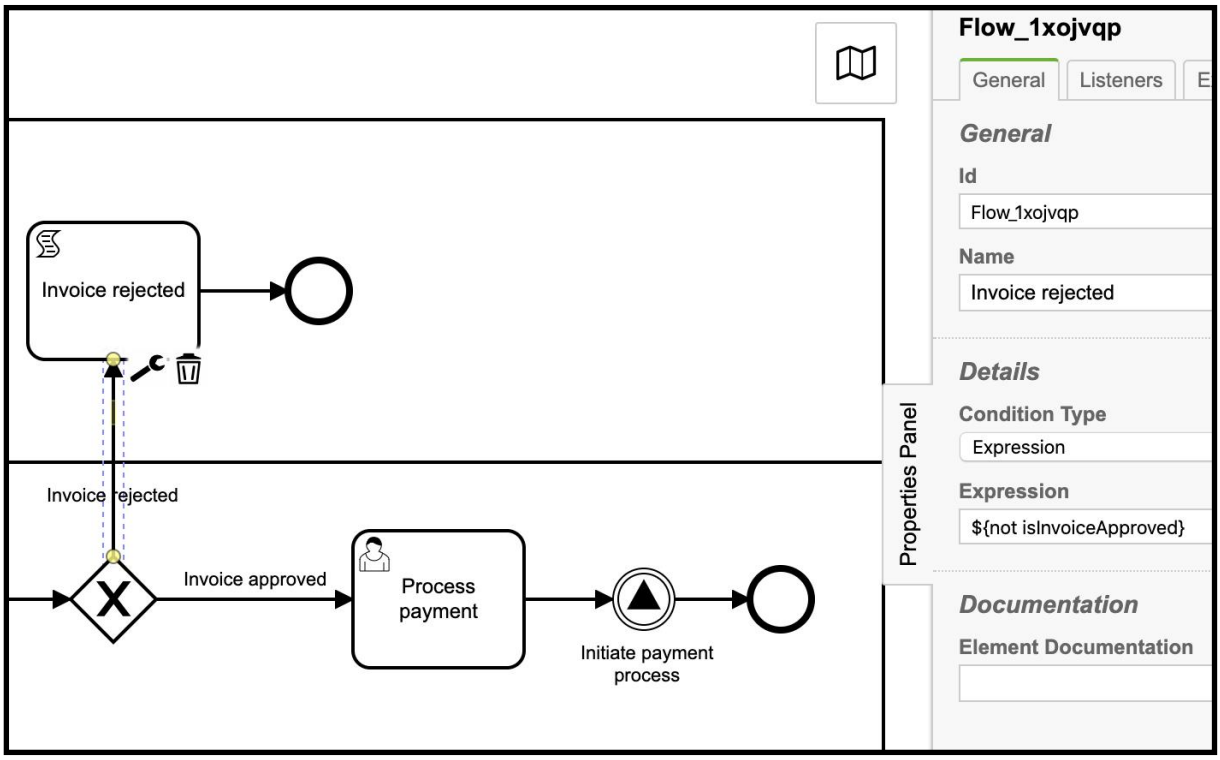


Id: Activity_0c4shmb
Name: Approve Invoice
Assignee: demo

Task form:

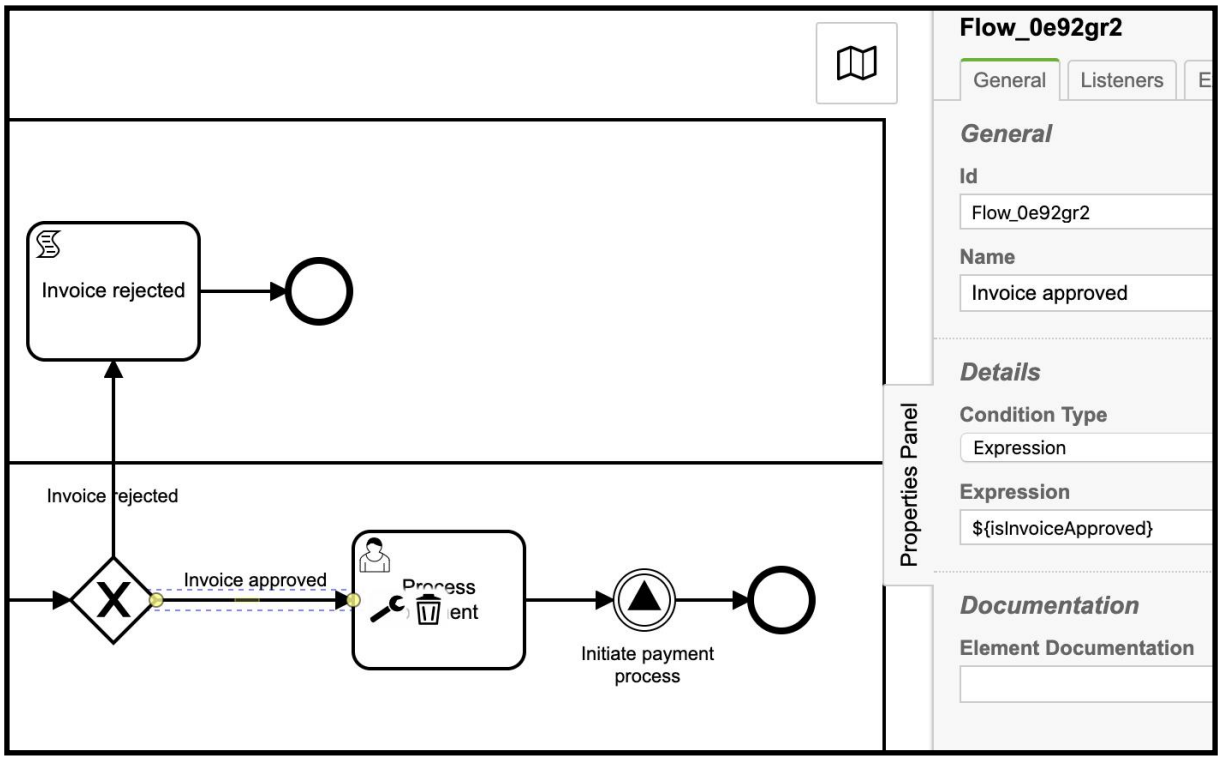


Sequence flow constraint: Invoice rejected



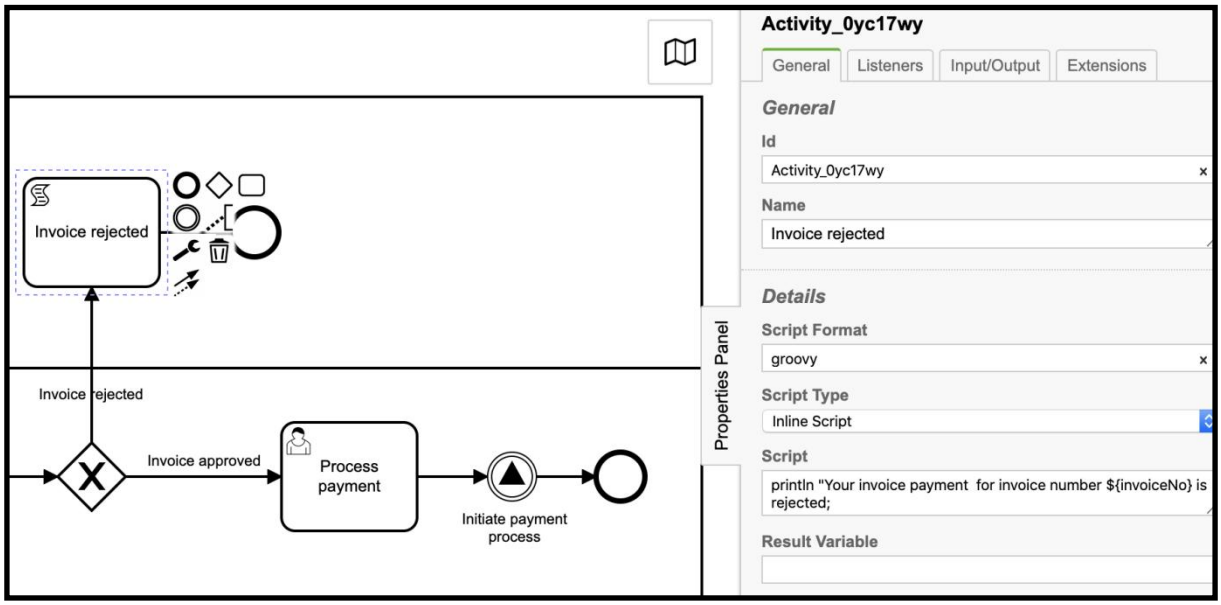
Id: Flow_1xojvqp
Name: Invoice rejected
Condition Type: Expression
Expression: ${not isInvoiceApproved}

Sequence flow constraint: Invoice approved



Id: Flow_0e92gr2
Name: Invoice approved
Condition Type: Expression
Expression: ${isInvoiceApproved}

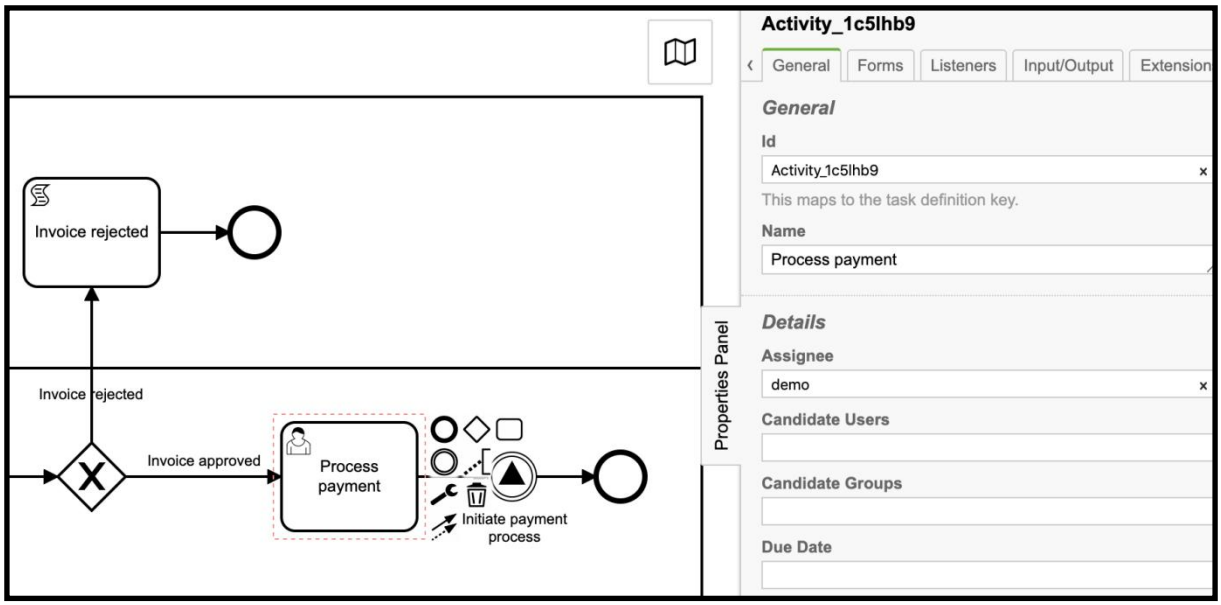
Script task : Invoice rejected



Id: Activity_0yc17wy
Name: Invoice rejected
Script Format: groovy
Script Type: Inline Script
Script:

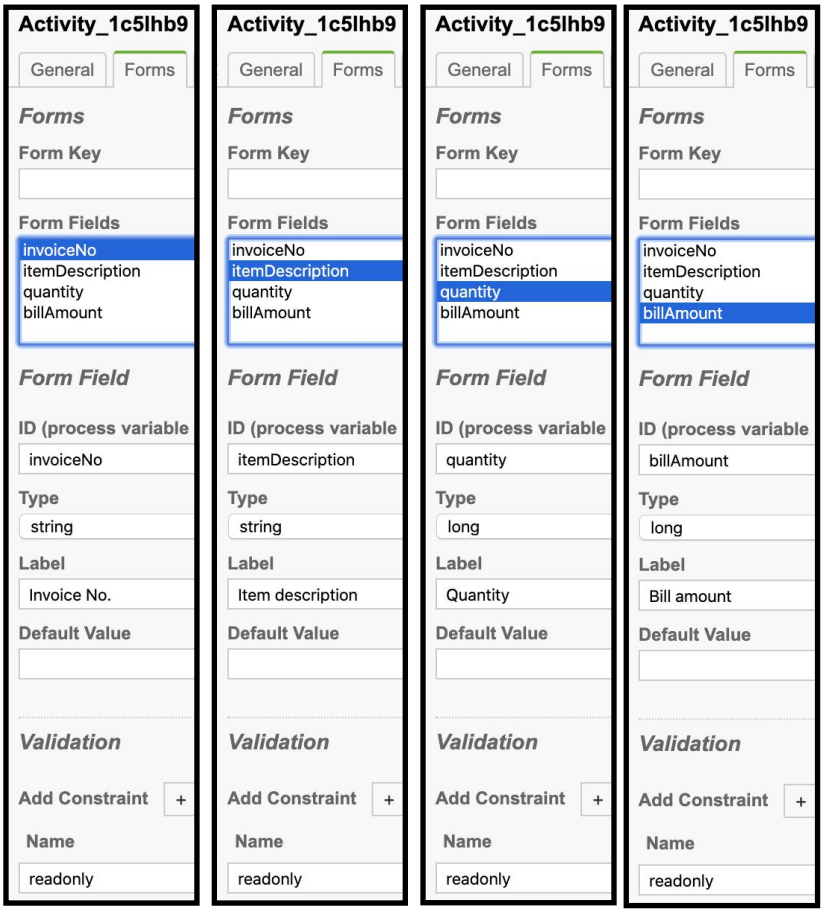
```
println "Your invoice payment  for invoice number ${invoiceNo} is rejected;
```

User task: Process payment

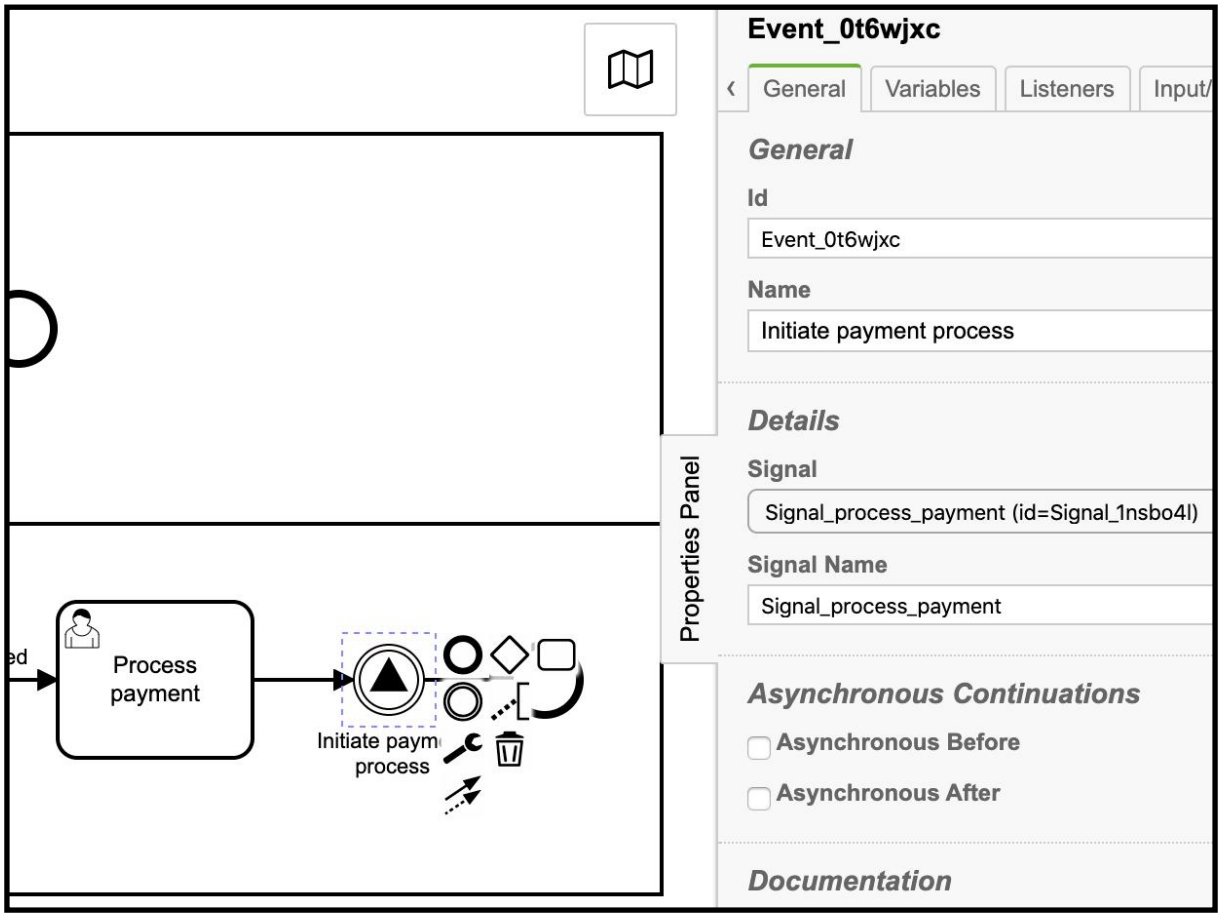


Id: Activity_1c5lhb9
Name: Process payment
Assignee: demo

Task form:

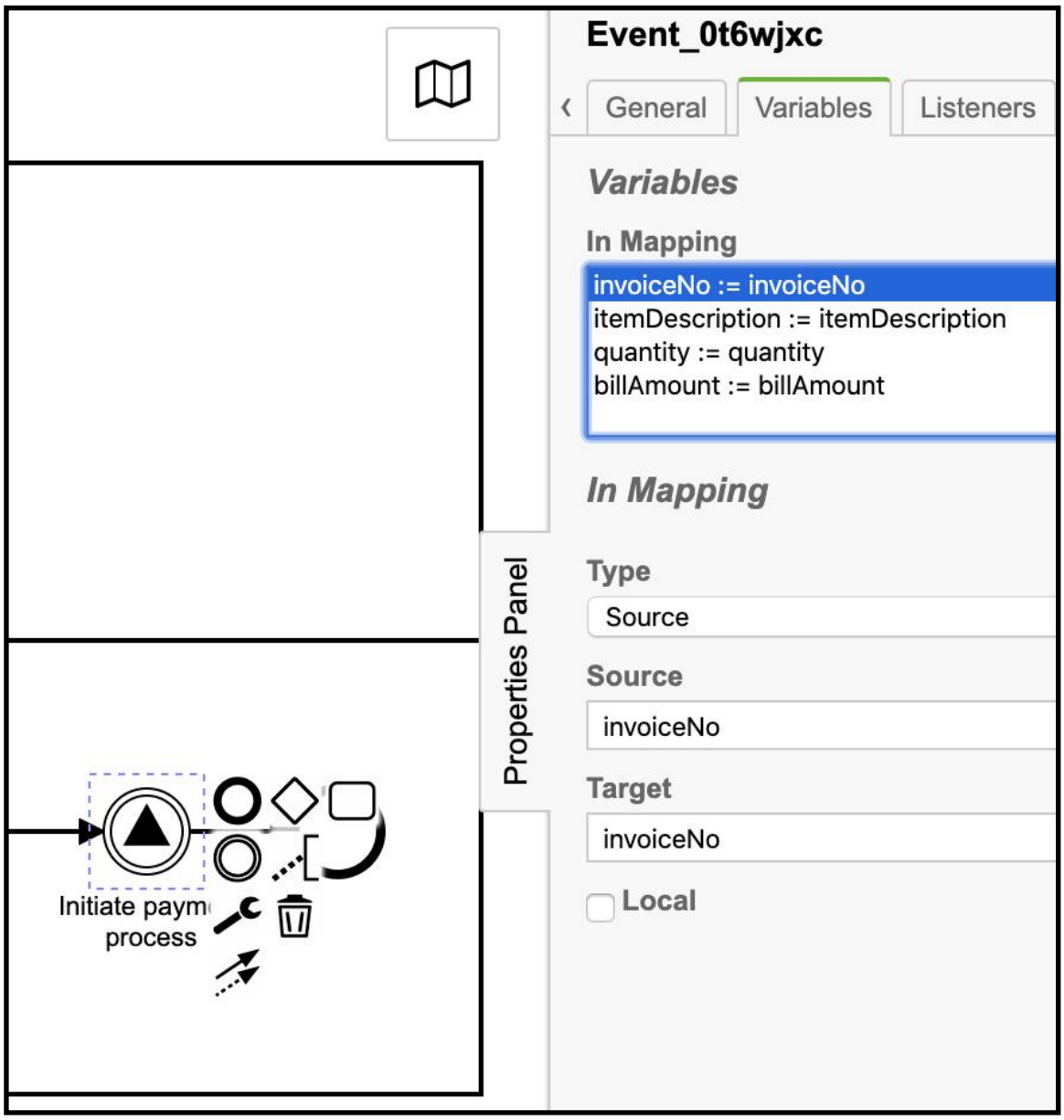


Intermediate signal throw event: Initiate payment process



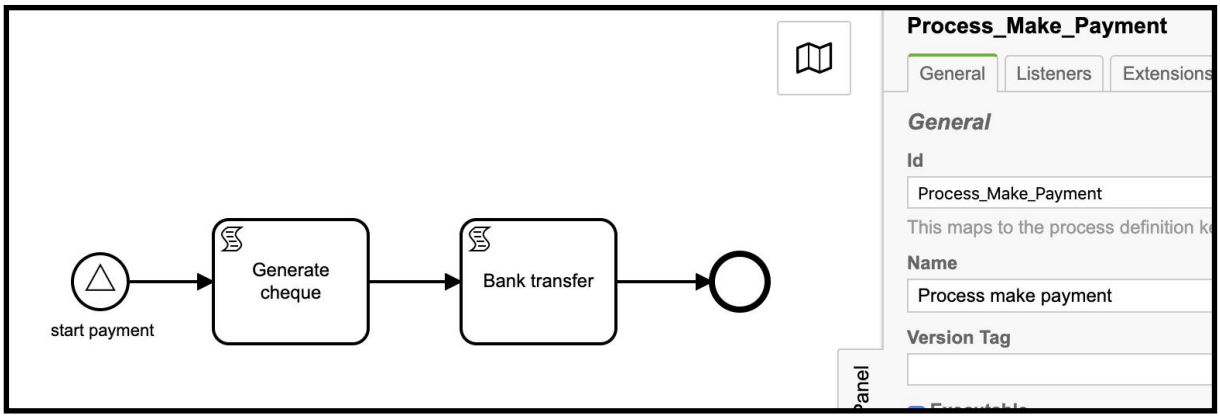
Id: Event_0t6wjxc
Name: Initiate payment process
Signal: Signal_1nsbo4l
Signal Name: Signal_process_payment

Signal Variables:



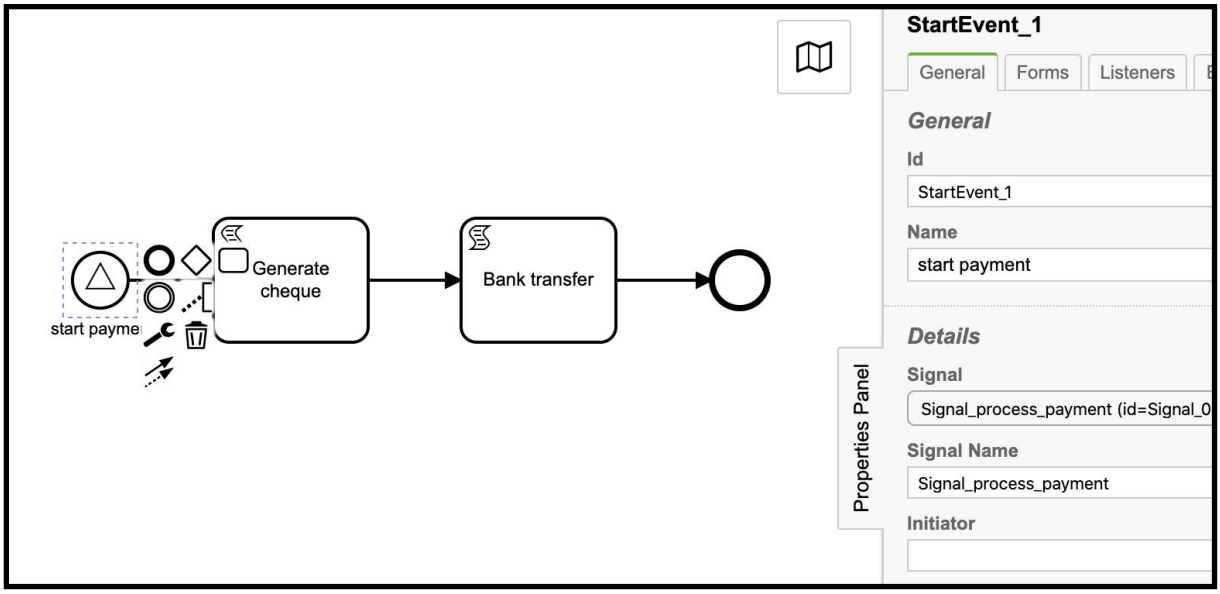
These variables will be passed along with thrown signal event.

2. Start signal event



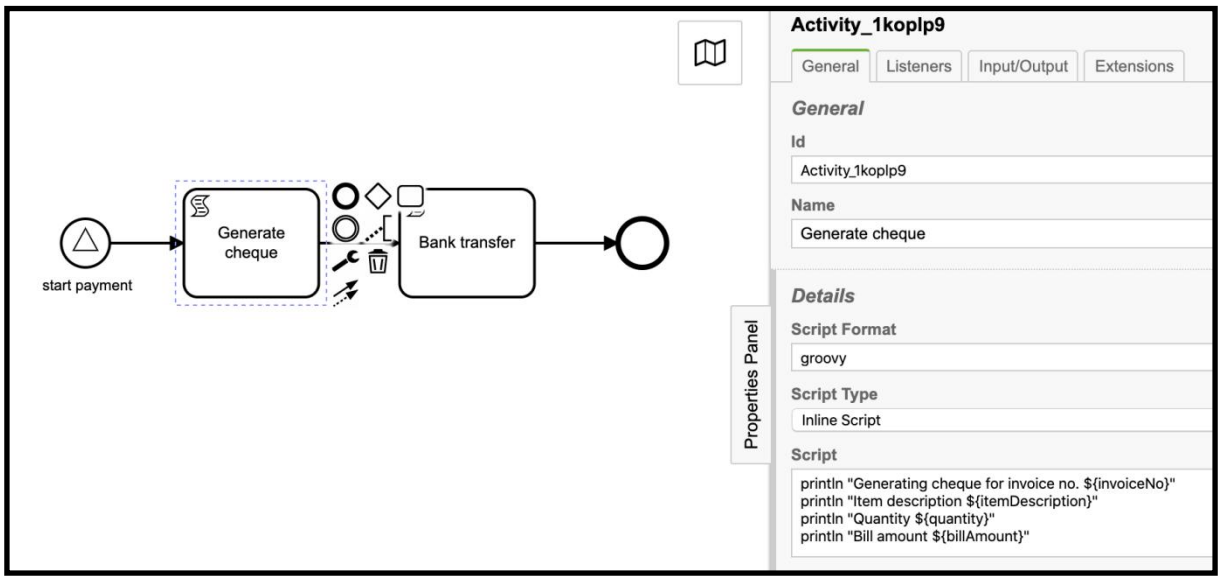
Id: Process_Make_Payment
Name: Process make payment

Start signal event:



Id: StartEvent_1
Name: start payment
Signal: Signal_0kpsnrg
Signal Name: Signal_process_payment

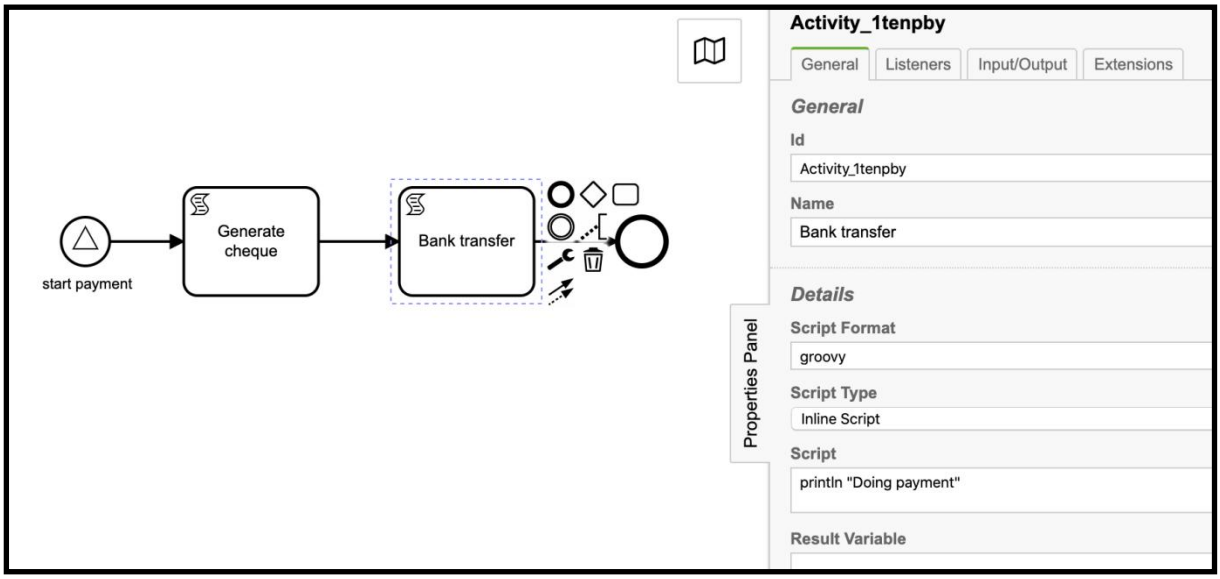
Script task: Generate cheque



Id: Activity_1koplp9
Name: Generate cheque
Script Format: groovy
Script Type: Inline Script
Script:

```
println "Generating cheque for invoice no. ${invoiceNo}"
println "Item description ${itemDescription}"
println "Quantity ${quantity}"
println "Bill amount ${billAmount}"
```

Script task: Generate cheque

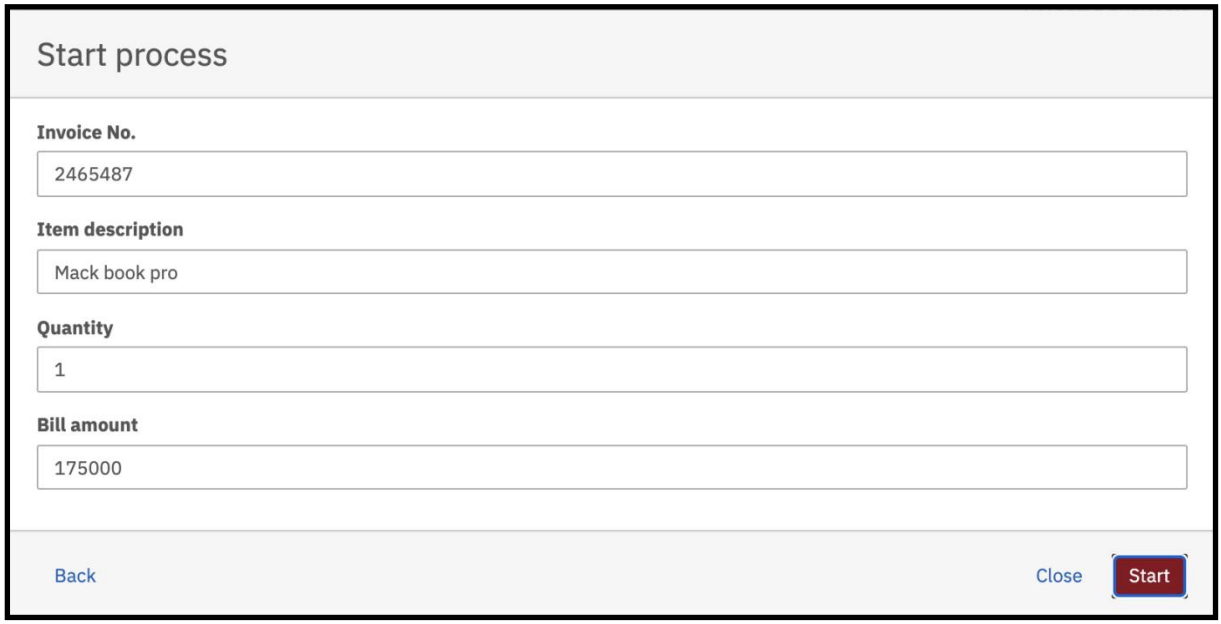


Id: Activity_1tenpby
Name: Bank transfer
Script Format: groovy
Script Type: Inline Script
Script:

```
println "Doing payment”
```

Deploy both the processes and start process Process payment invoice

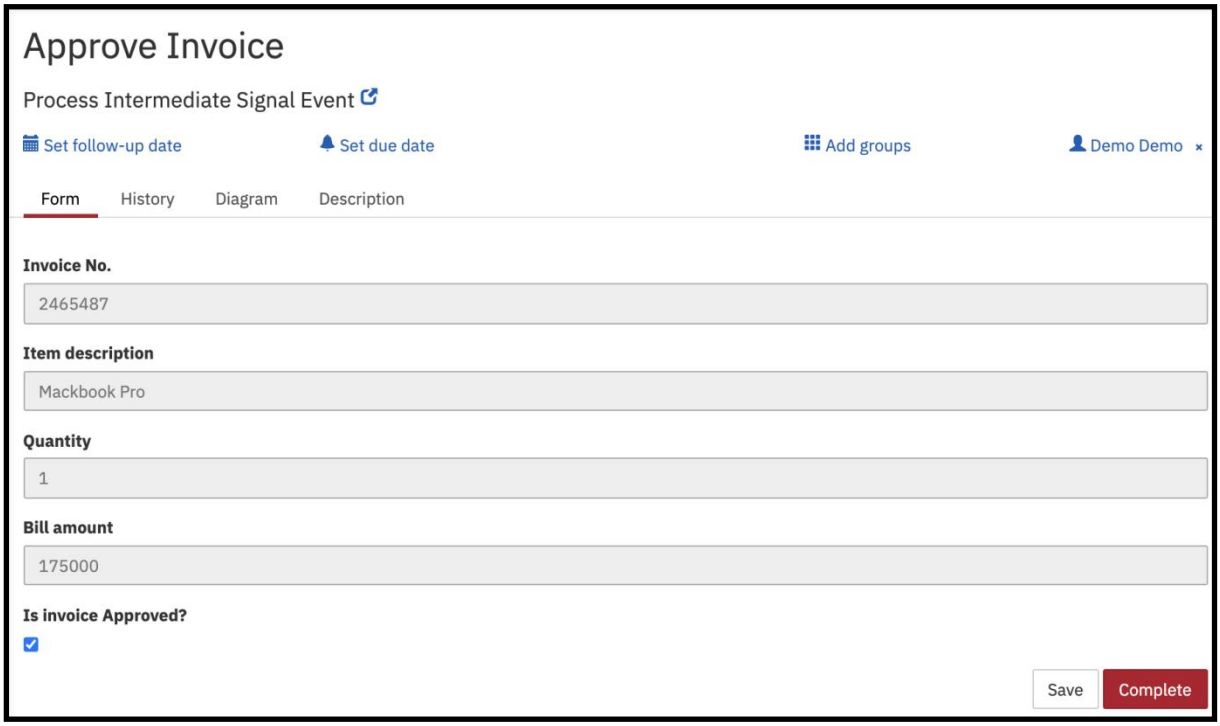
Hit the Process Intermediate Signal Event. It will open start process form to initiate process.



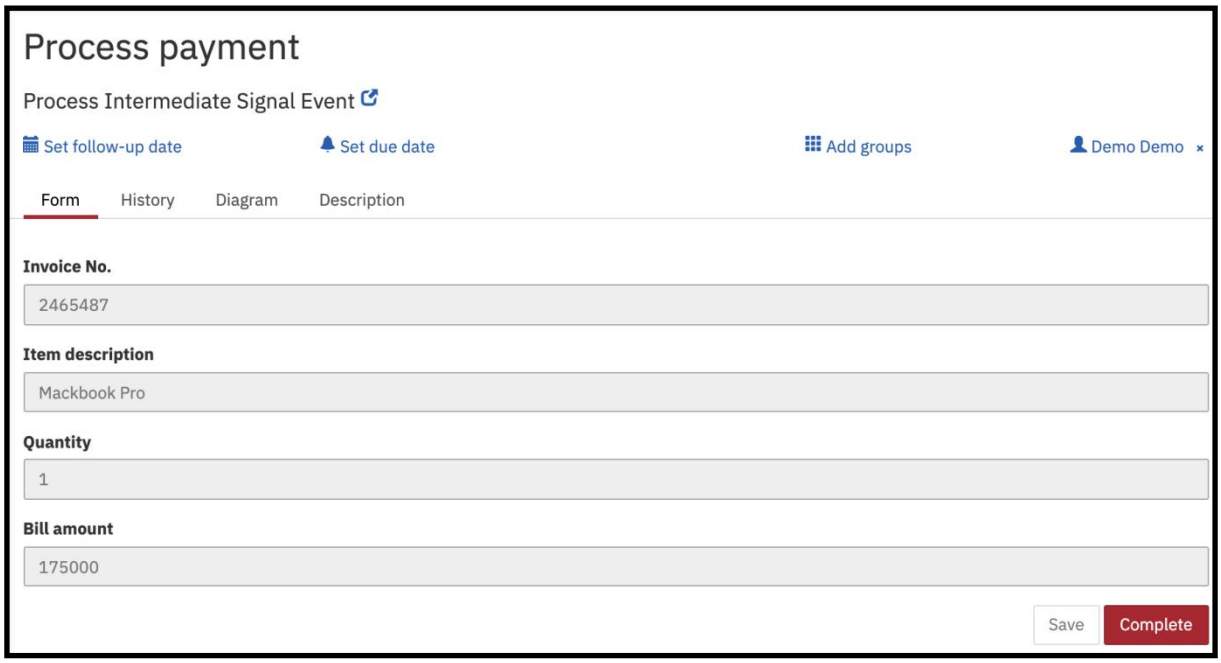
Hit the start process.

As per process definition Receive invoice user task will get created.

Hit the Complete button and as per process definition **Approve Invoice** user task will get created.



Checked Is Invoice Approved? Form field and hit the Complete button and as per process definition Process Payment user task will get created.

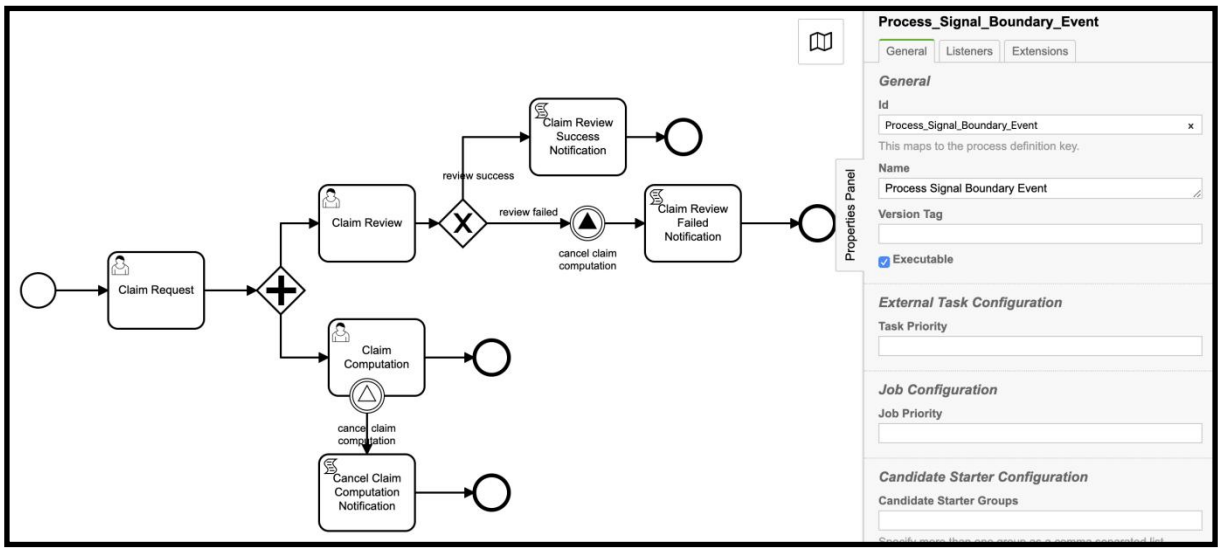


Hit the Complete button and as per process definition intermediate signal throw event will get fired with invoice related data and Process make payment will get initiated.

To verify the Process make payment executed on trigger of intermediate throw signal event observe the tomcat console log.

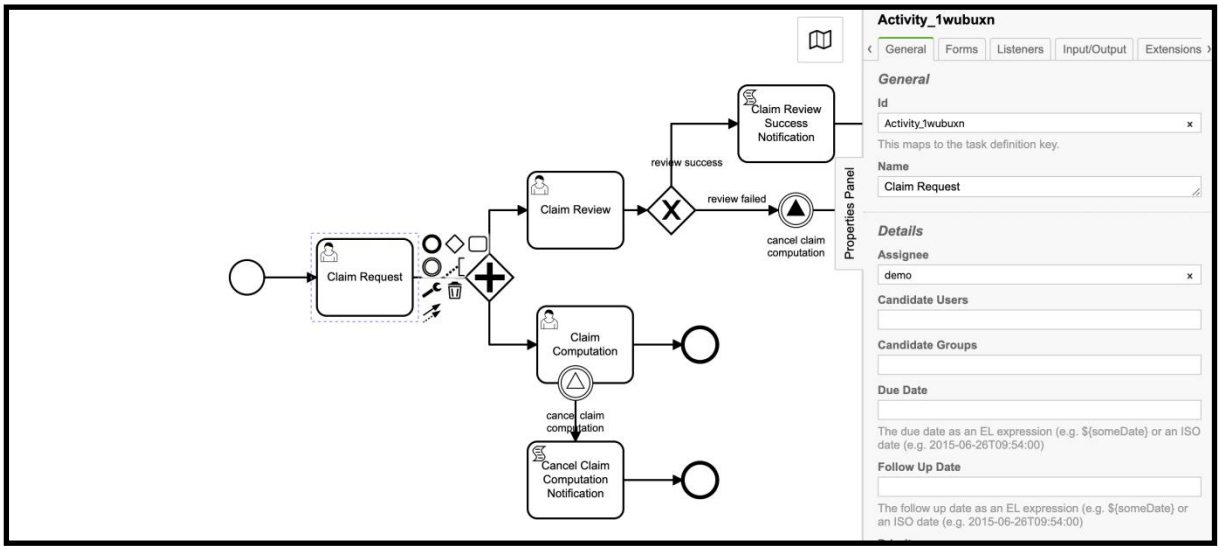
Generating cheque for invoice no. 2465487
Item description Mack book pro
Quantity 1
Bill amount 175000
Doing payment

3. Boundary signal event



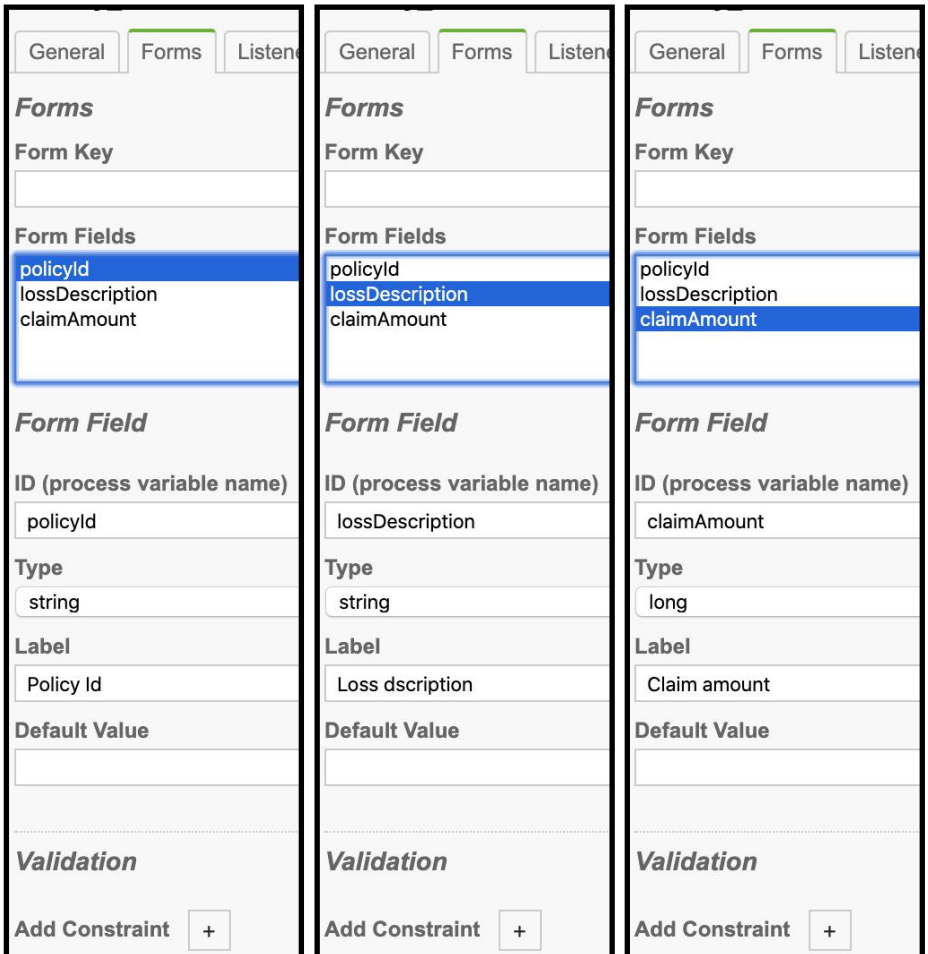
Id: Process_Signal_Boundary_Event
Name: Process Signal Boundary Event

User task: Claim Request



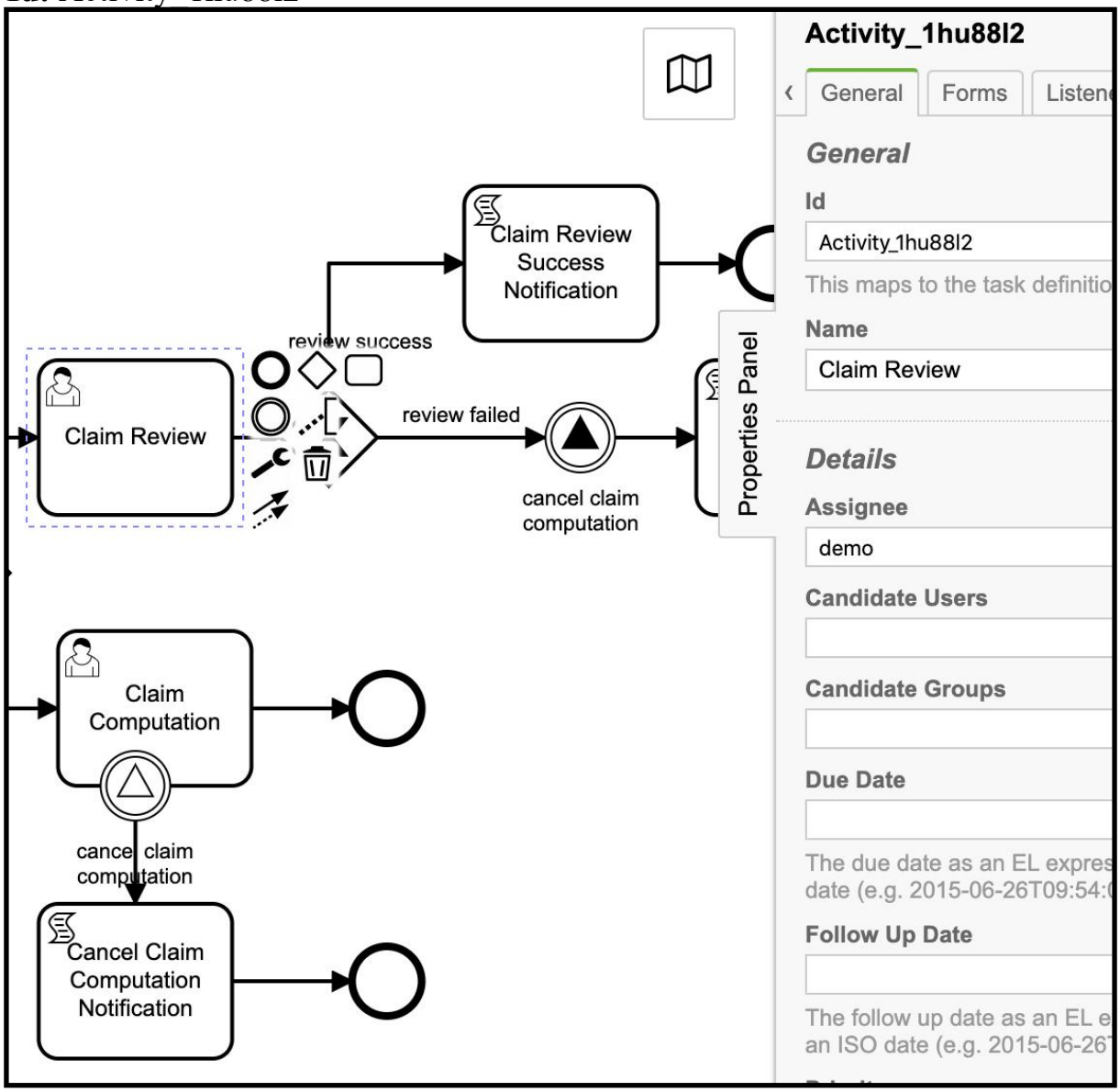
Id: Activity_1wubuxn
Name: Claim Request
Assignee: demo

Task form:



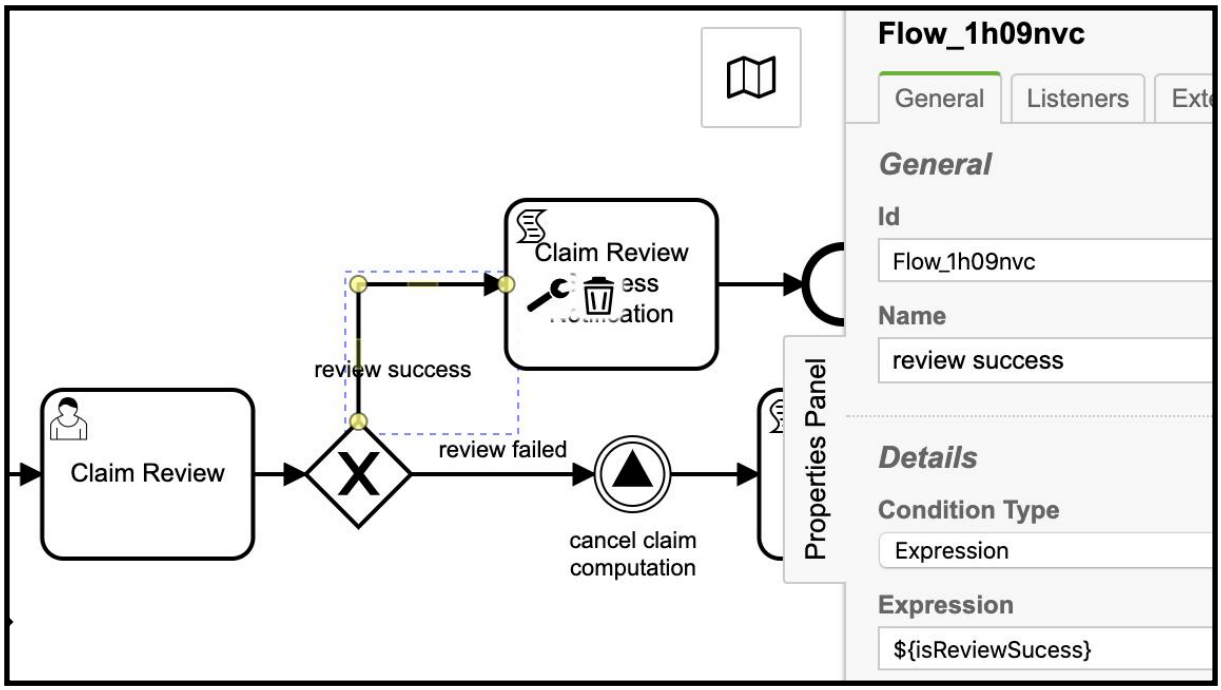
User task: Claim Review

Id: Activity_1hu88l2



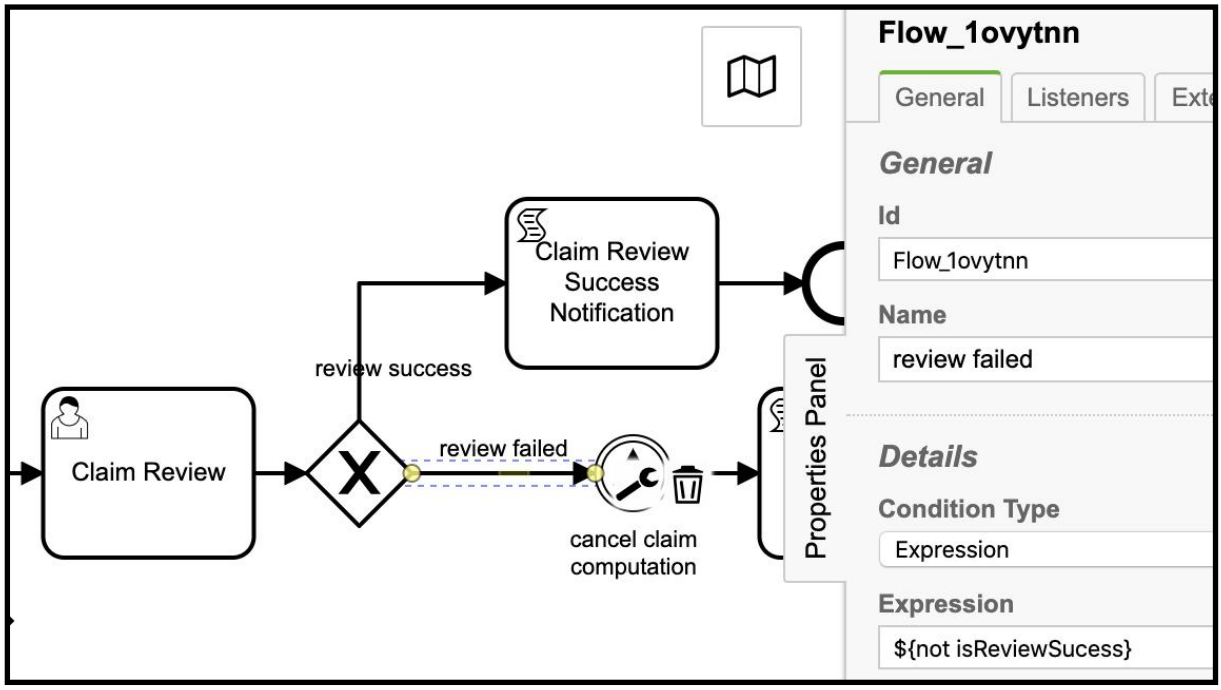
Name: Claim Review
Assignee: demo

Sequence flow constraint: review success



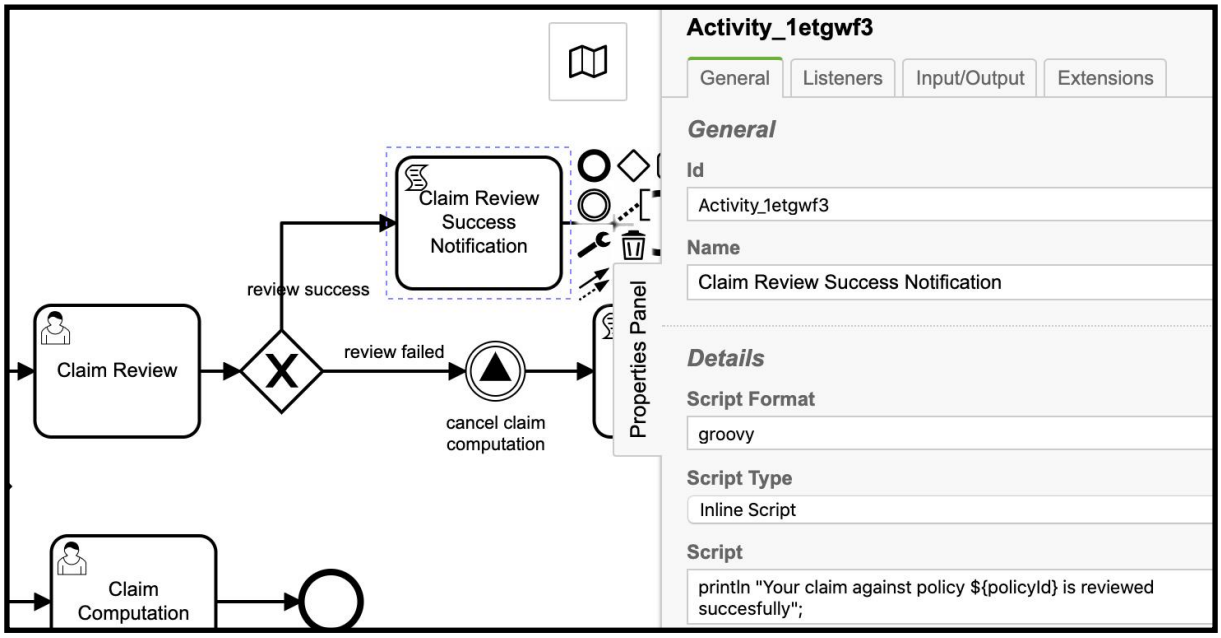
Id: Flow_1h09nvc
Name: review success
Condition Type: Express
Expression: ${isReviewSucess}

Sequence flow constraint: review failed



Id: Flow_1ovytnn
Name: review failed
Condition Type: Express
Expression: ${not isReviewSucess}

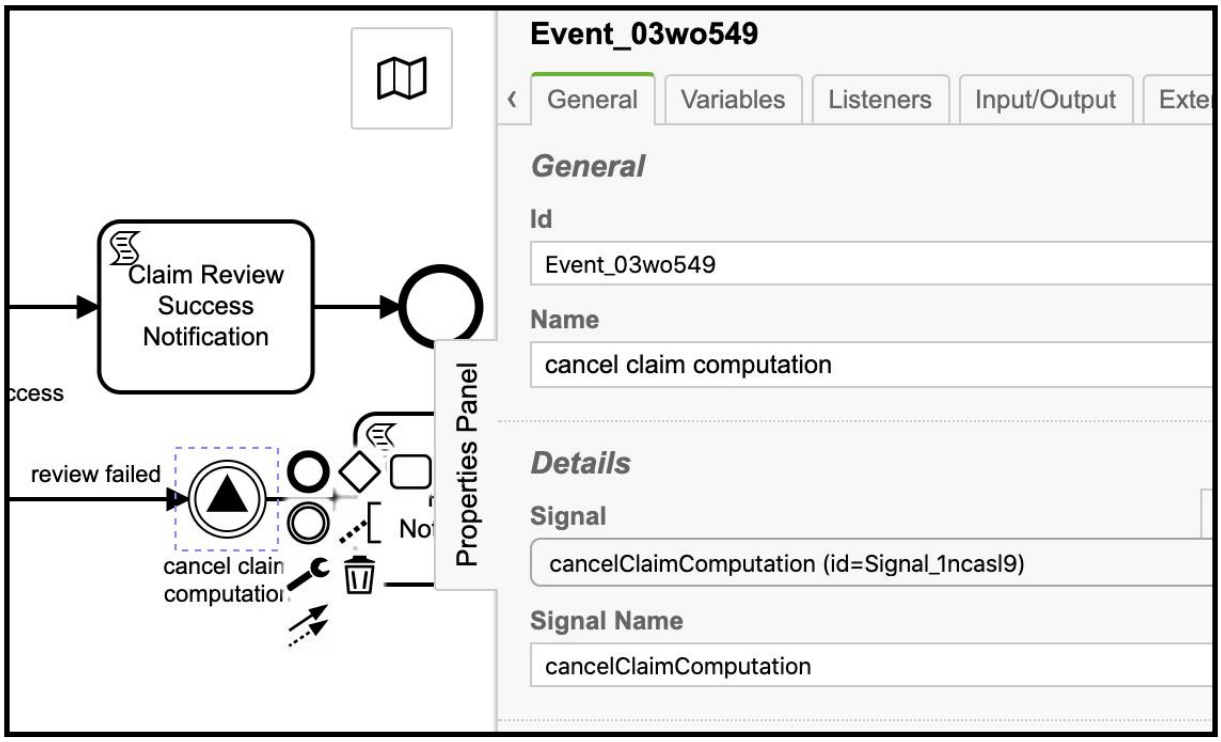
Script task: Claim Review Success Notification



Id: Activity_1etgwf3
Name: Claim Review Success Notification
Script Format: groovy
Script Type: Inline Script
Script:

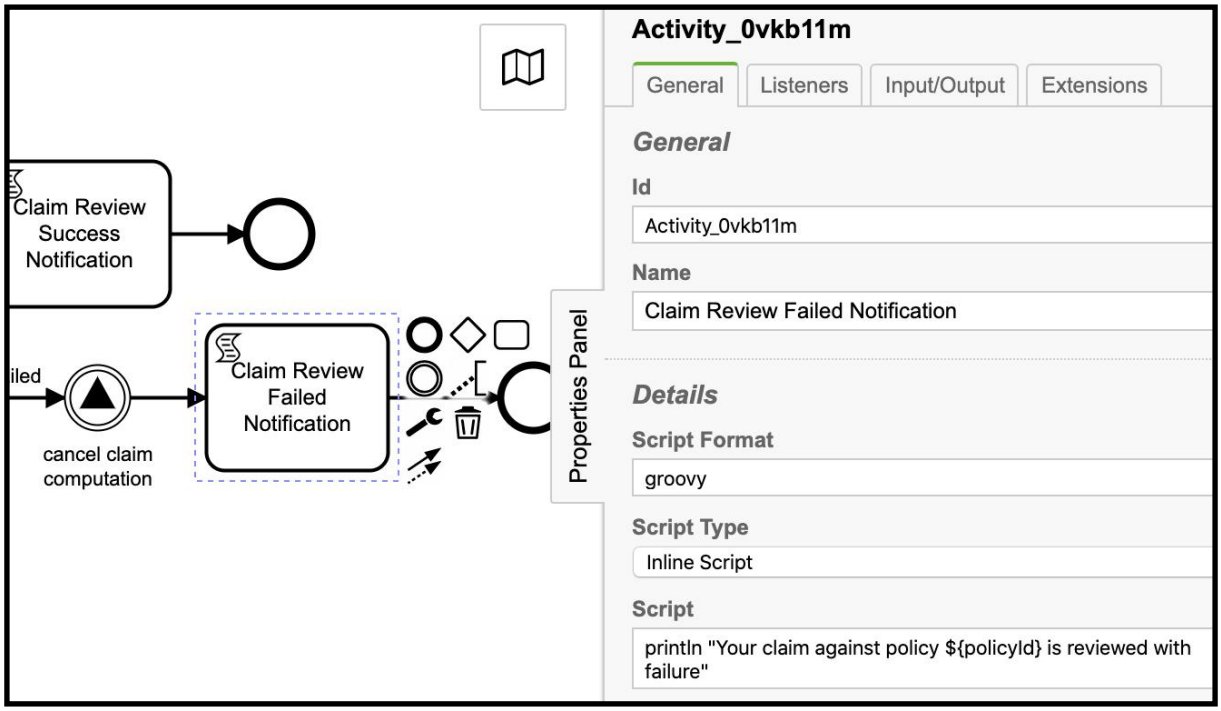
```
println "Your claim against policy ${policyId} is reviewed succesfully";
```

Intermediate signal throw event: cancel claim computation



Id: Event_03wo549
Name: cancel claim computation
Signal: Signal_1ncasl9
Signal Name: cancelClaimComputation

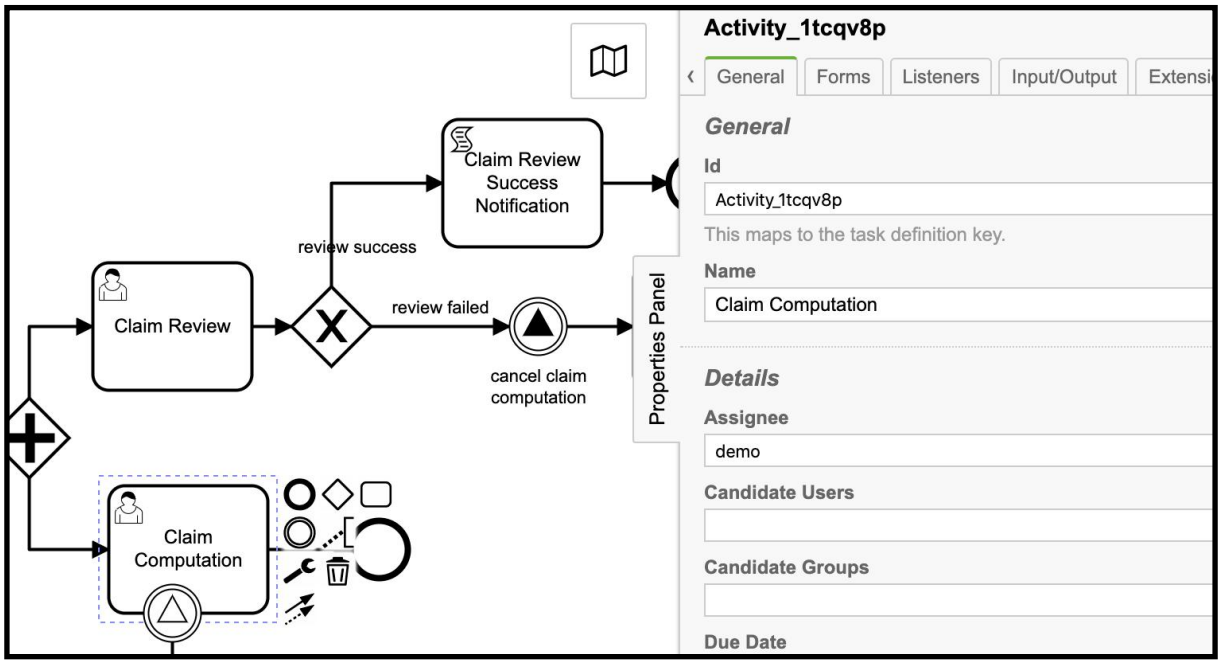
Script task: Claim Review Failed Notification



Id: Activity_0vkb11m
Name: Claim Review Failed Notification
Script Format: groovy
Script Type: Inline Script
Script:

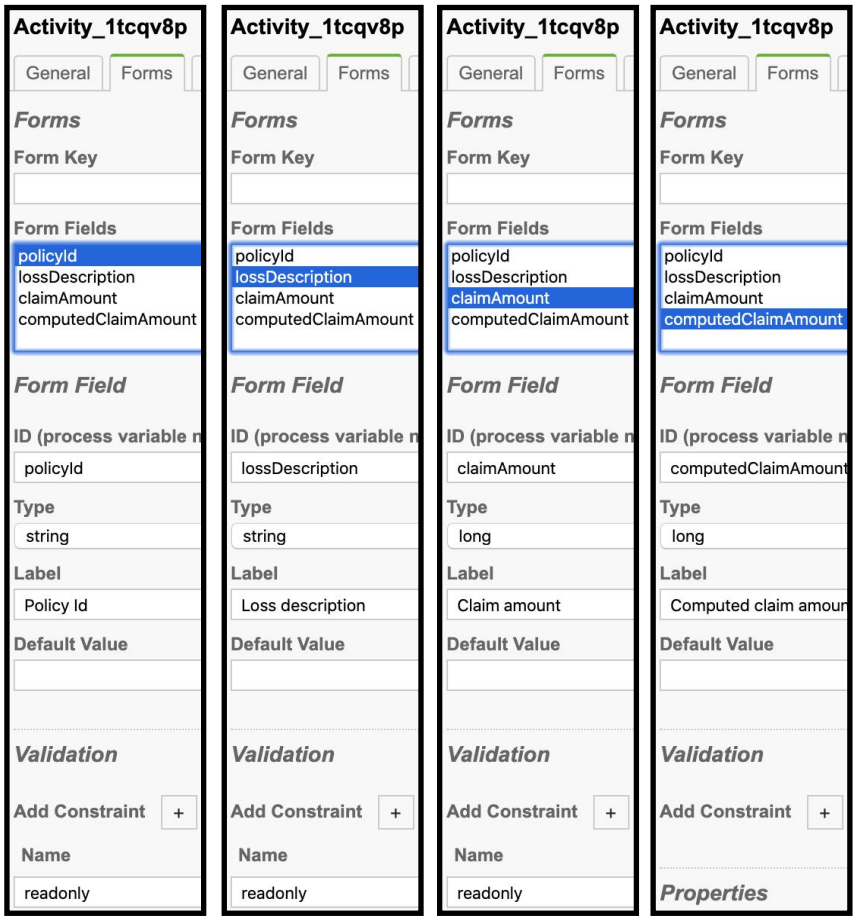
```
println "Your claim against policy ${policyId} is reviewed with failure”
```

User task: Claim Computation

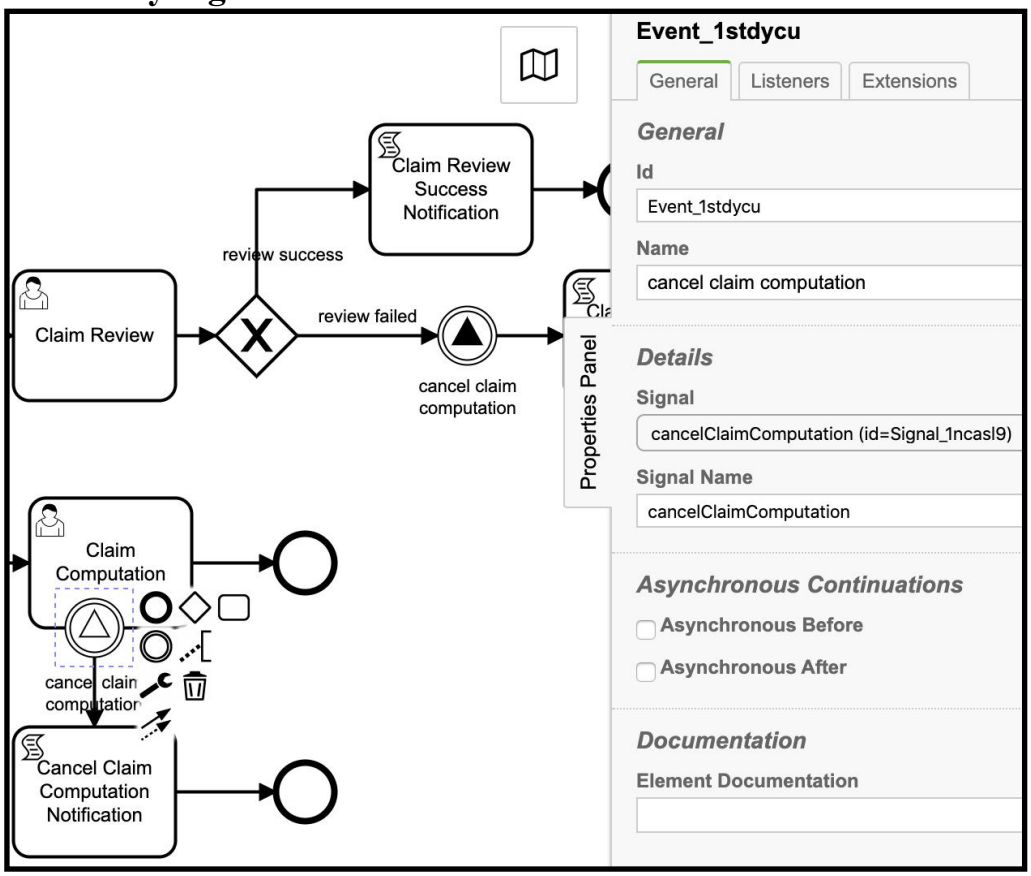


Id: Activity_1tcqv8p
Name: Claim Computation
Assignee: demo

Task form:



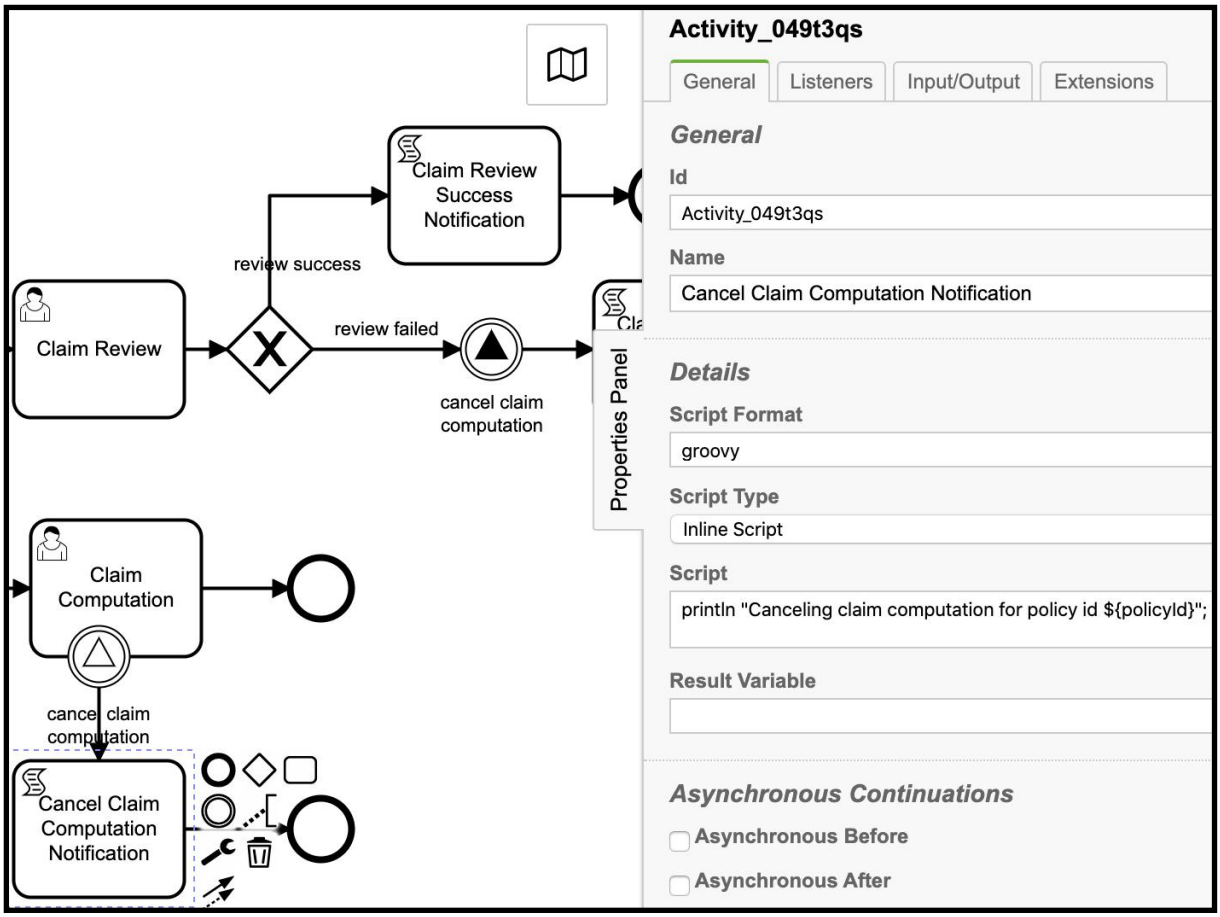
Boundary Signal event:



Id:

Event_1stdycu
Name: cancel claim computation
Signal: Signal_1ncasl9
Signal Name: cancelClaimComputation

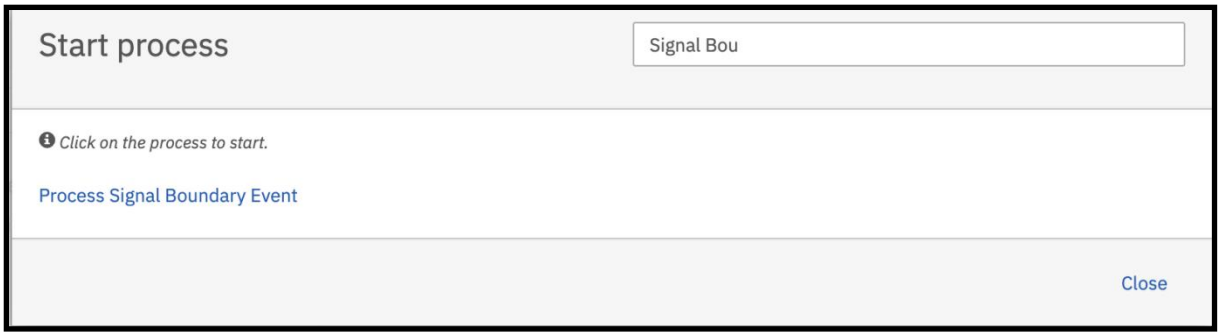
Script task: Cancel Claim Computation Notification



Id: Activity_049t3qs
Name: Cancel Claim Computation Notification
Script Format: groovy
Script Type: Inline Script
Script:

```
println "Canceling claim computation for policy id ${policyId}";
```

Deploy and start the process



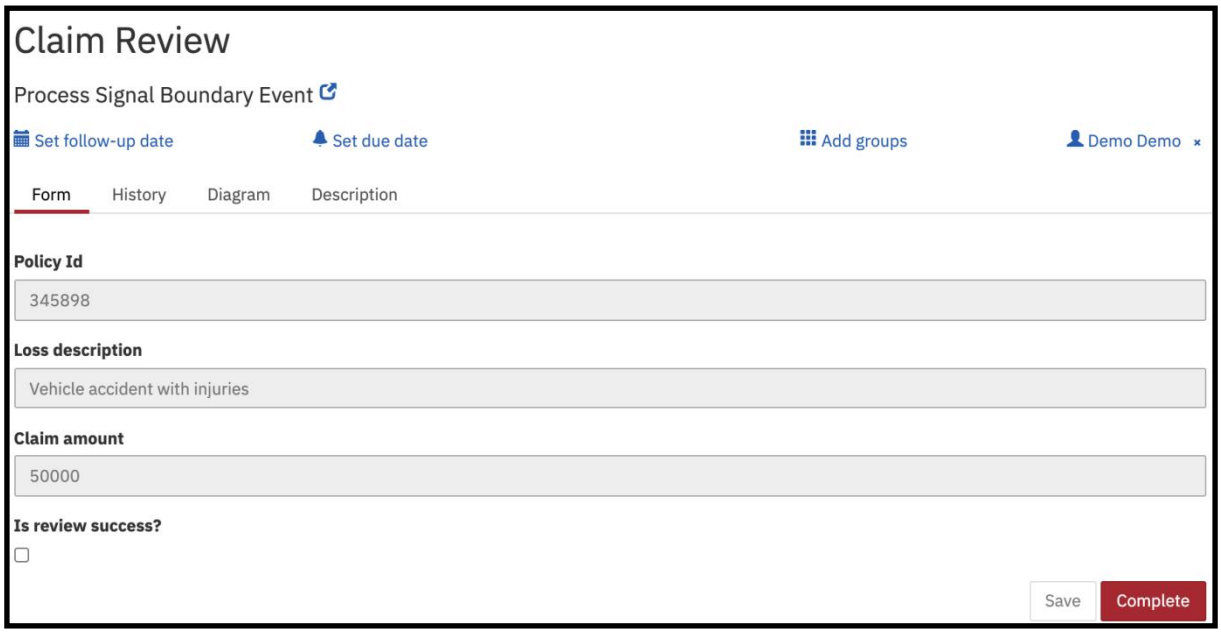
Hit the Process Signal Boundary Event and then hit the Start button to start process.

As per process definition Claim Request user task will get created.

Hit the Complete button.

As per process definition, two user tasks Claim Review and Claim Computation will get created as we have used parallel gateway.

Open the Claim Review task.



Keep Is review success field unchecked and Hit the Complete button so the intermediate signal throw event to cancel claim computation get triggered and same signal will be caught by boundary signal event on Claim computation user task.

Observe the tomcat console

Your claim against policy 345898 is reviewed with failure
Canceling claim computation for policy id 345898